UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Project Report

Project Title : SecureGov Framework: An Integrated Security, Governance and Risk Analysis Platform

Course title : Security Governance, Risk Management and Compliance Sessional

Course code : SEC 116

Submitted by :	Submitted to:
Faria Yasmin Jui (2404036) Momtasin Tamann (2404012) Anika Akter nira (2404034) Level : 01 Term : 02 Department of Cyber Security Engineering, UFTB	Rakib Hossen Assistant Professor, Department of Cyber Security Engineering, UFTB

SecureGov Framework : An Integrated Security, Governance & Risk Analysis Platform

Introduction

The SecureGov Framework is a unified platform that brings security, governance, and risk management into one place for government organizations. Usually, these things are handled separately, which creates huge blind spots and makes it hard to track cyber threats quickly. SecureGov fixes this by linking network scanning, vulnerability detection, and exploit testing into a single workflow. Built inside Kali Linux, it uses popular open-source tools to monitor network health in real time. The main goal is simple: take complicated system logs and turn them into clear, actionable compliance updates for decision-makers.

Objectives

- Real-time security monitoring and risk visibility.
- Automate compliance assessment and report generation.
- Detect and prioritize security vulnerabilities.
- Centralize security findings into a unified dashboard.
- Reduce manual work and human errors.
- Enable proactive risk identification and mitigation.
- Support vulnerability lifecycle management.
- Generate comprehensive executive and technical reports.

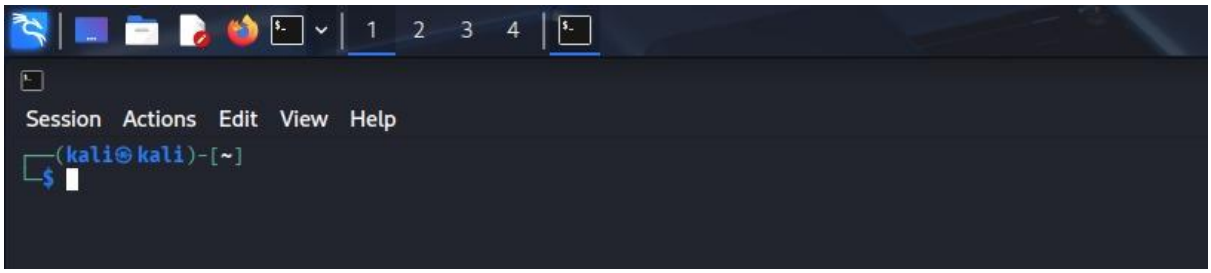
Tools

- **Kali Linux:** Primary operating system used for the entire security testing environment.
- **Nmap:** Asset discovery and network mapping tool.
- **OpenVAS:** Vulnerability assessment and risk detection tool.
- **Wireshark:** Packet inspection and traffic analysis utility.
- **Metasploit:** Vulnerability and controlled exploitation testing framework.

Implementation Steps:

Kali Linux

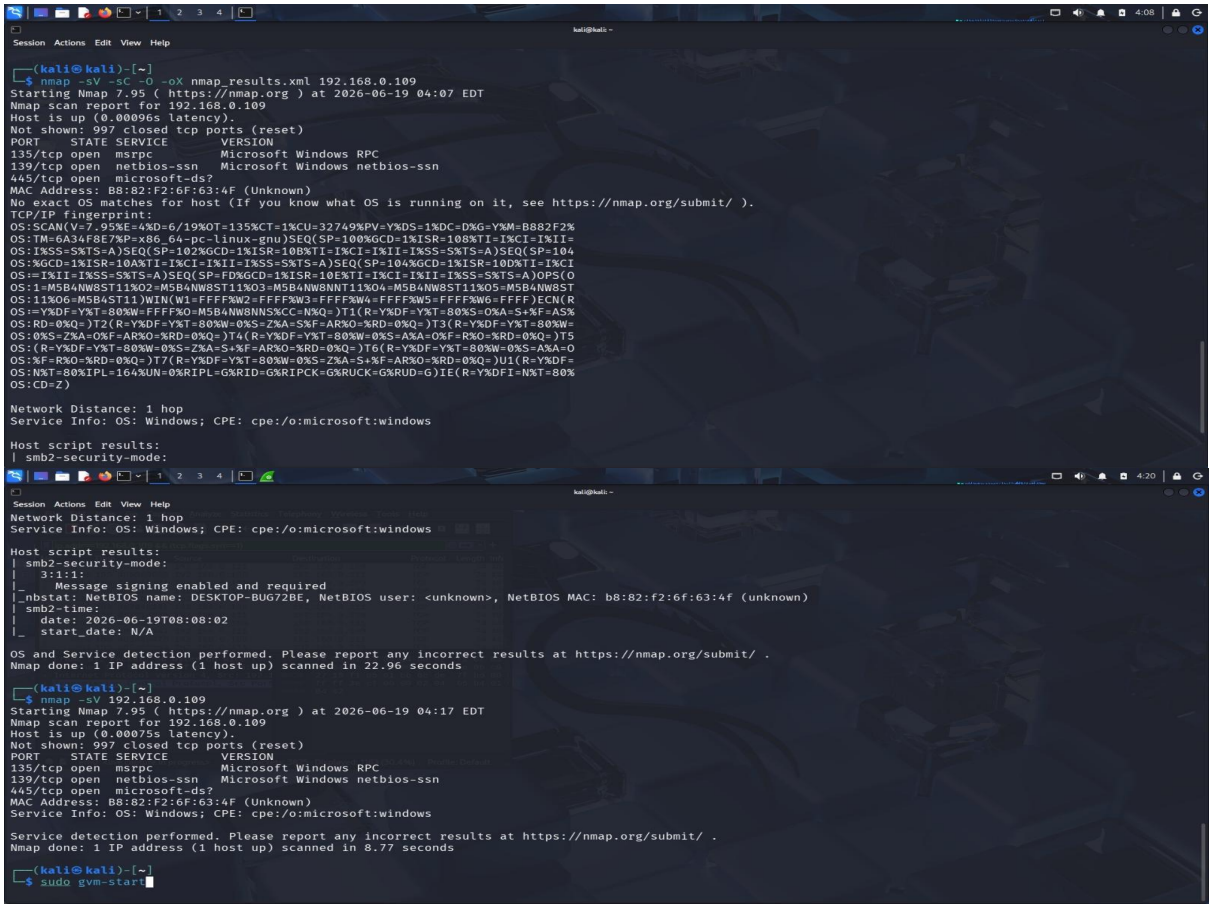
Open Kali Linux Terminal First, run your Kali Linux OS. Open the terminal and ensure your system is updated.

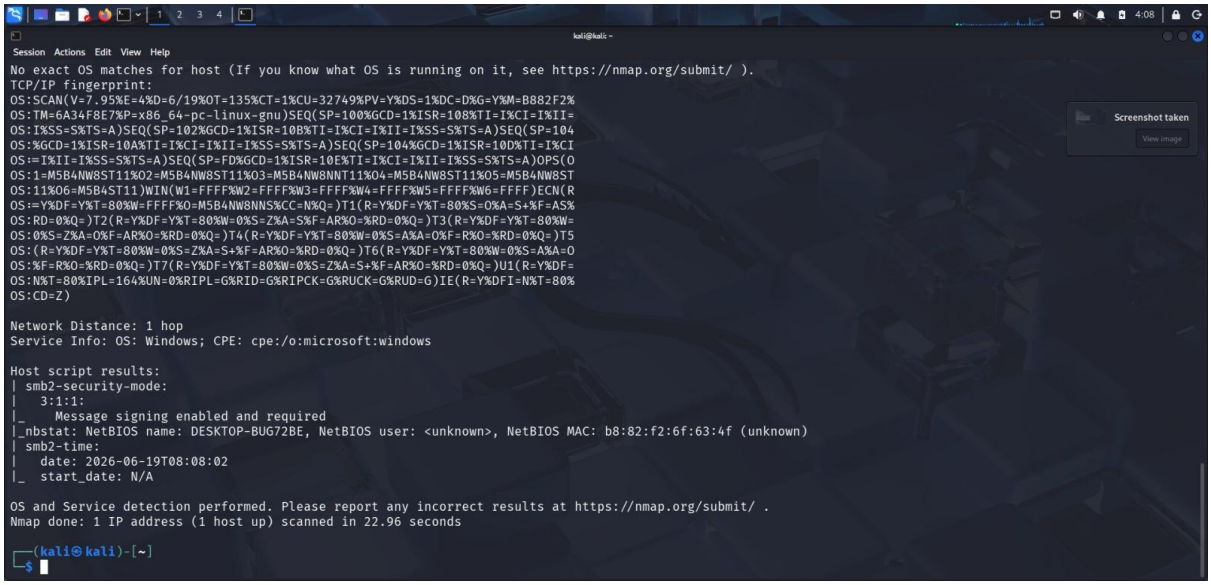


Reconnaissance and Network Scanning (Nmap)

To identify whether the target host (**192.168.0.109**) is online and determine which ports and services are running, an aggressive Nmap scan was performed.

nmap -sV -sC -O -oX nmap_results.xml 192.168.0.109



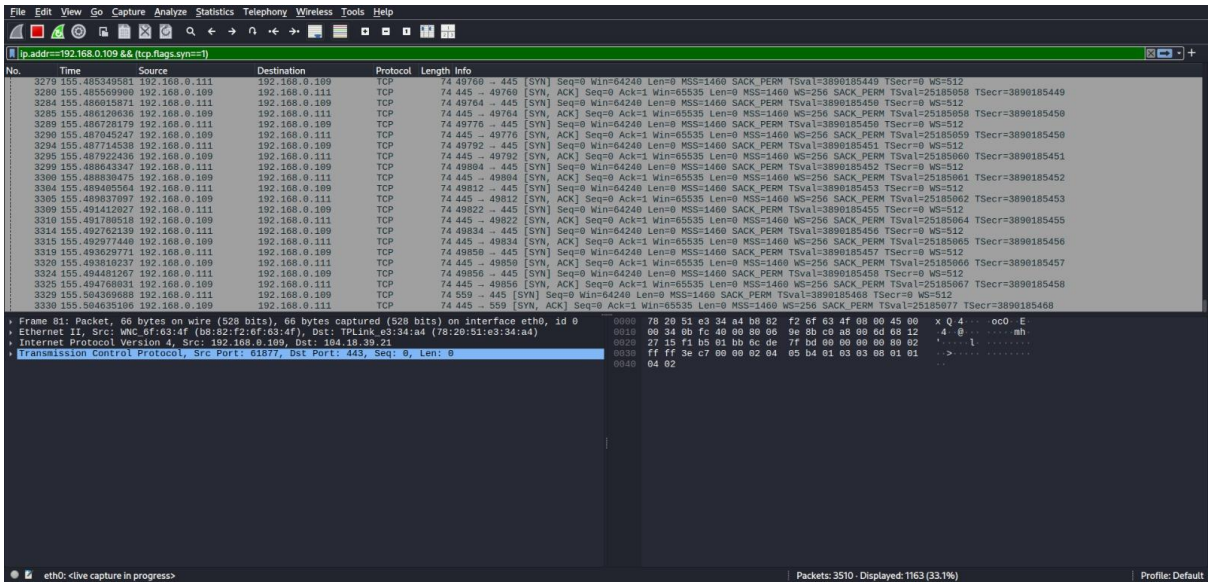


Traffic Capture & Handshake Analysis (Wireshark)

While the Nmap scan was running, Wireshark was used to monitor the network traffic and observe the TCP Three-Way Handshake between the scanner and the target host.

Wireshark Display Filter:

ip.addr == 192.168.0.109 && (tcp.flags.syn == 1)

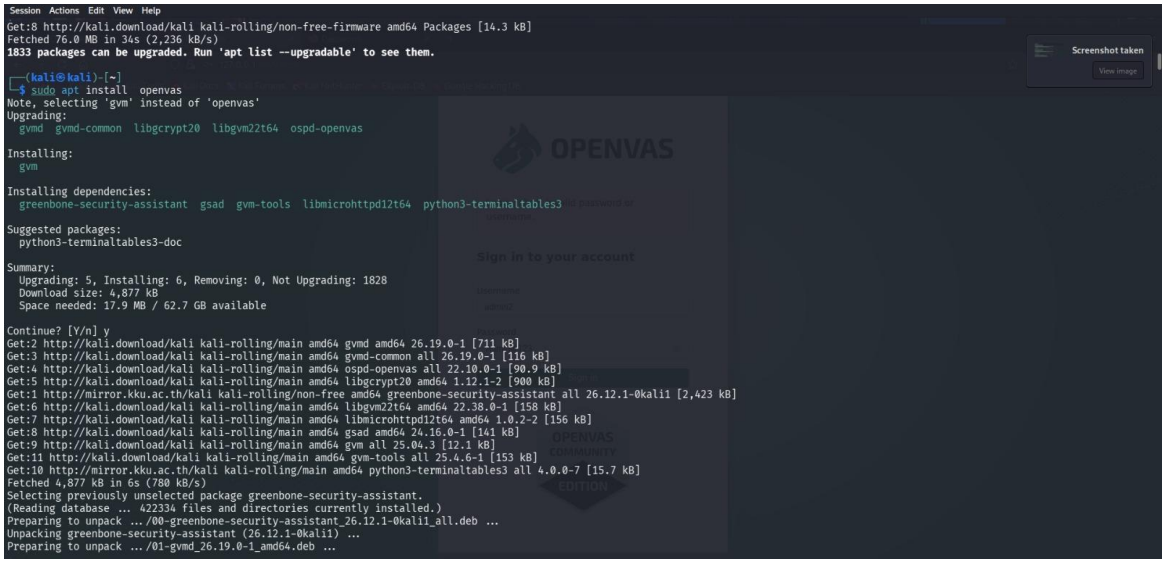


Automated Vulnerability Assessment (OpenVAS / GVM)

Step-1: Package Installation

The installation of **OpenVAS (Greenbone Vulnerability Manager - GVM)** begins by executing the following command in the terminal:

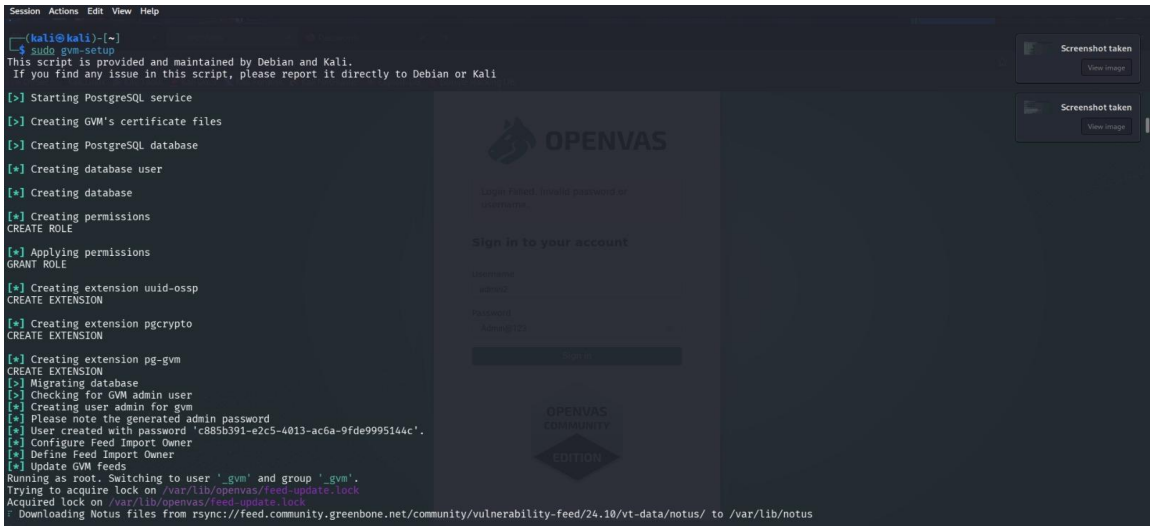
sudo apt install openvas



Step-2: Initial Setup & User Creation

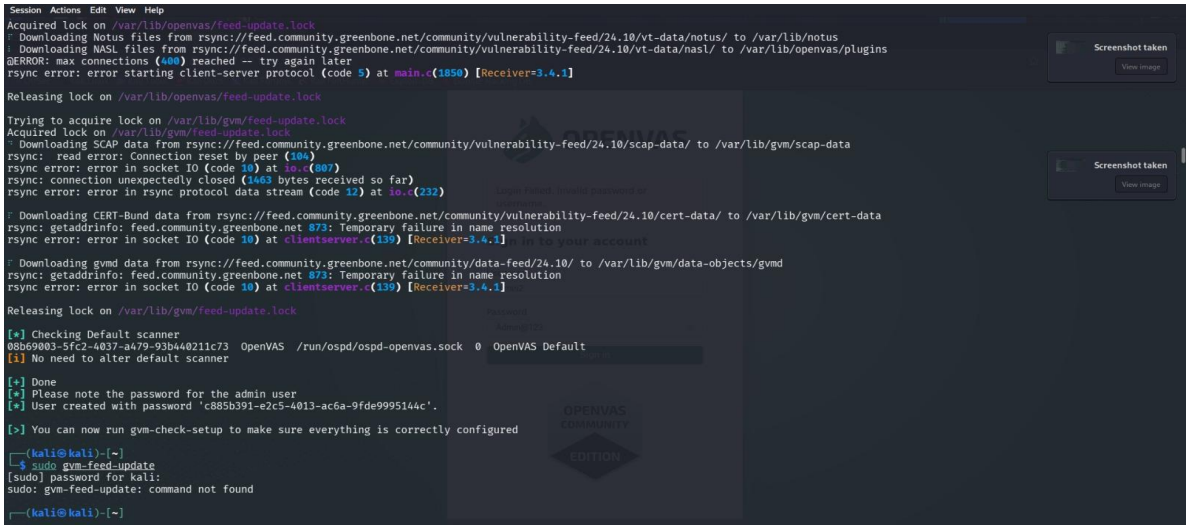
After the installation is complete, the following command is executed to perform the initial setup of GVM:

sudo gvm-setup



Step-3: Feed Update Errors & Troubleshooting

This stage shows network synchronization errors (rsync connection resets) during the vulnerability data stream update process. It also displays a "command not found" error when attempting an invalid sudo gvm-feed-update command.



Step-4: Feed Synchronization

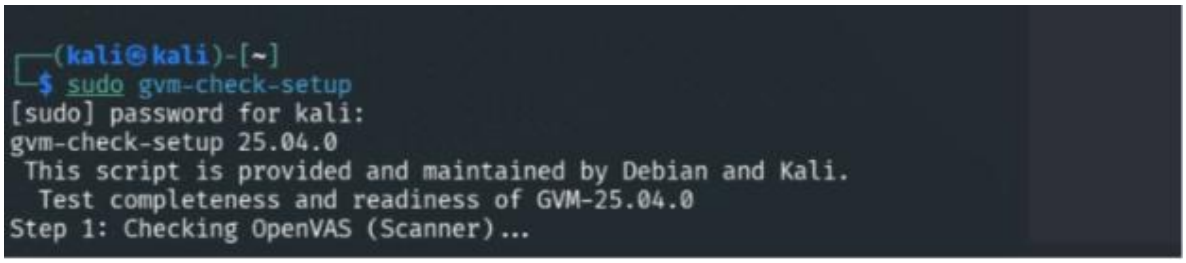
Before performing any vulnerability assessment, the vulnerability database and security signatures must be updated using the following command:

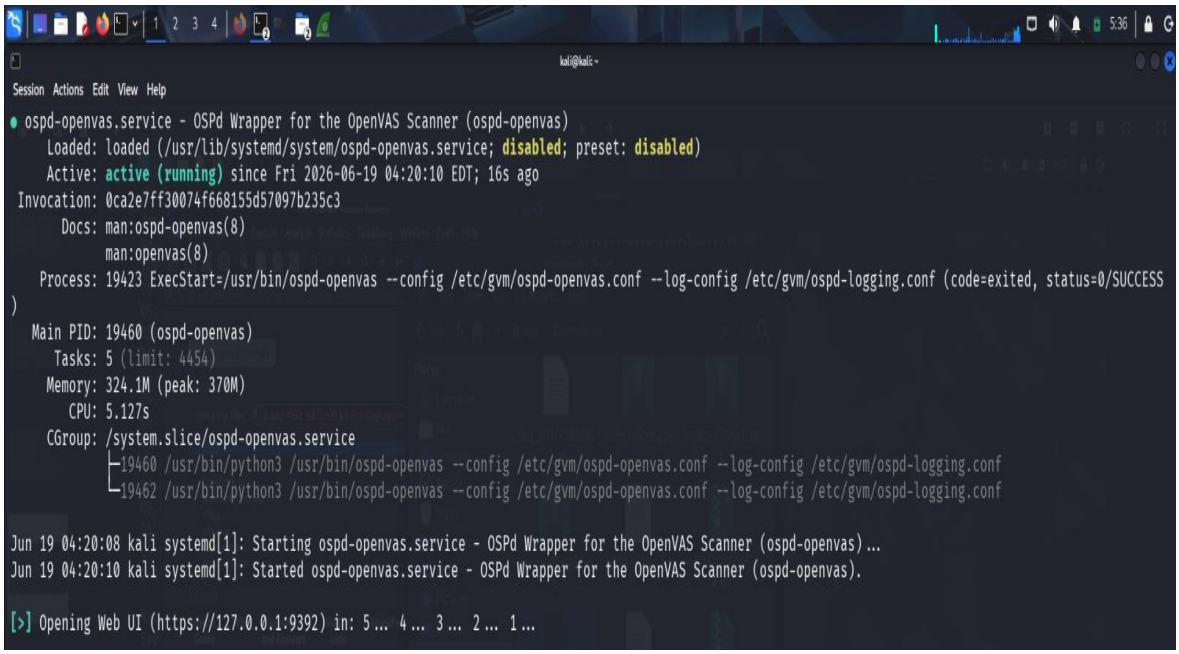
sudo greenbone-feed-sync

Step-5: Setup Verification

After the synchronization process is completed, the following command is executed to verify that all required services and configurations are functioning correctly:

sudo gvm-check-setup



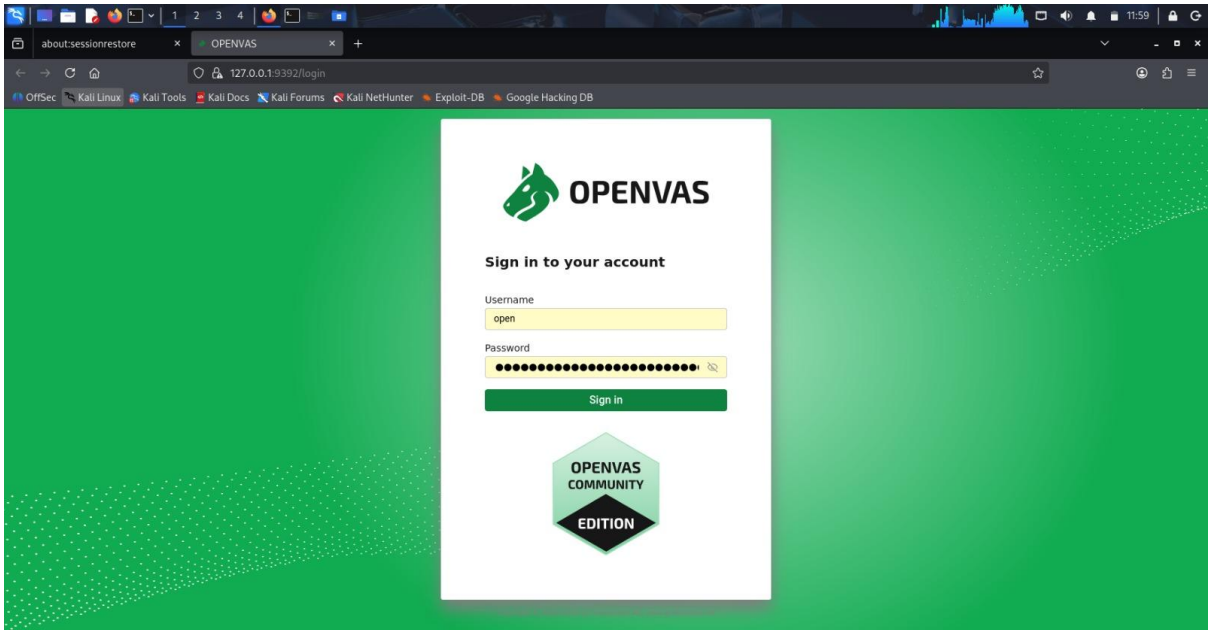


sudo runuser -u _gvm --gvmd --create-user=<username> --new-password=<password>

Step-6: Successful Web Portal Login

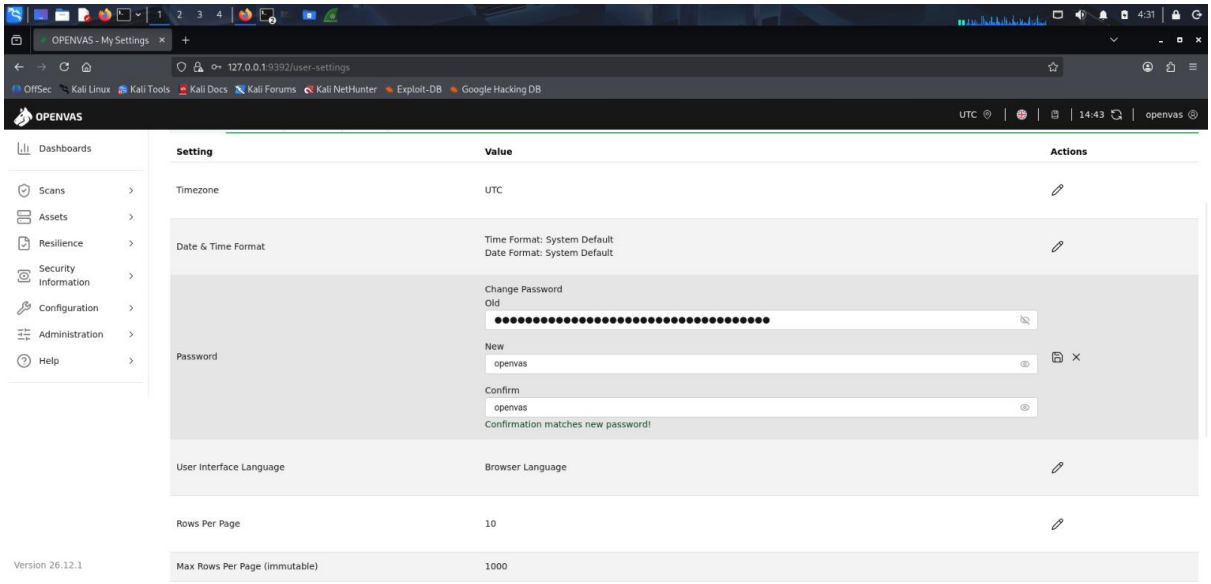
Once the setup and feed synchronization are completed successfully, the web management portal can be accessed by opening a web browser and navigating to:

https://localhost:9392



Step-7: Password Modification (User Settings)

The **My Settings** page accessed immediately after logging in to change the complex, auto-generated default password to a preferred custom password.

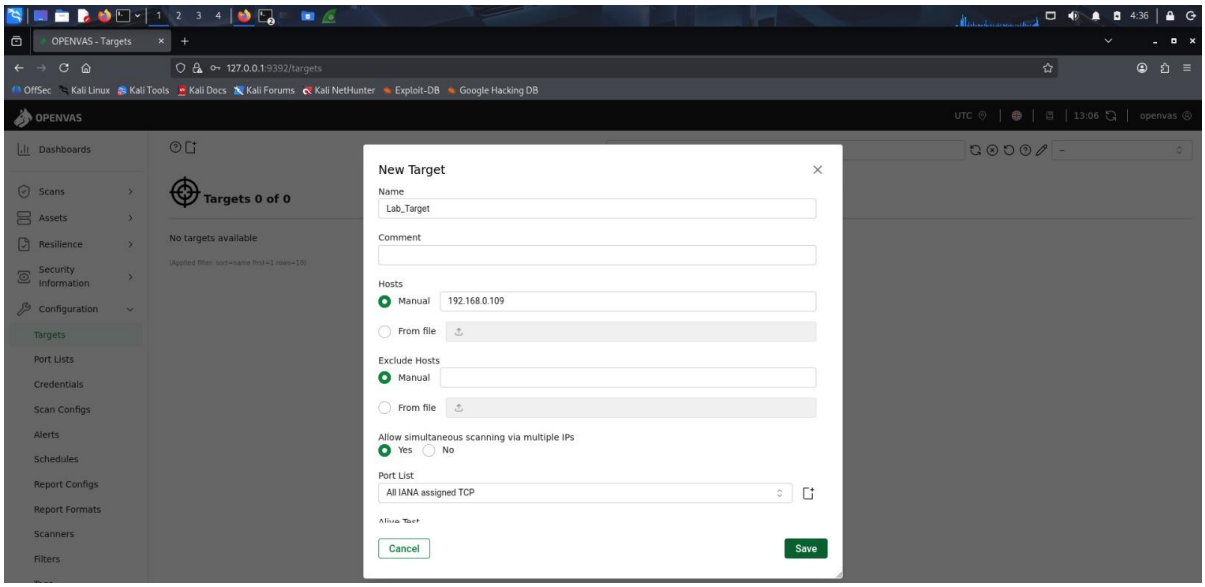


Step-8: Scan Target Configuration

Before launching a vulnerability scan, the target host must be configured within the GVM dashboard.

Configuration Steps:

- Navigate to **Configuration → Targets**.
- Create a new target.
- Enter the following information:
  - **Target Name:** Lab_Target
  - **Host IP Address:** 192.168.0.109
- Save the target configuration.

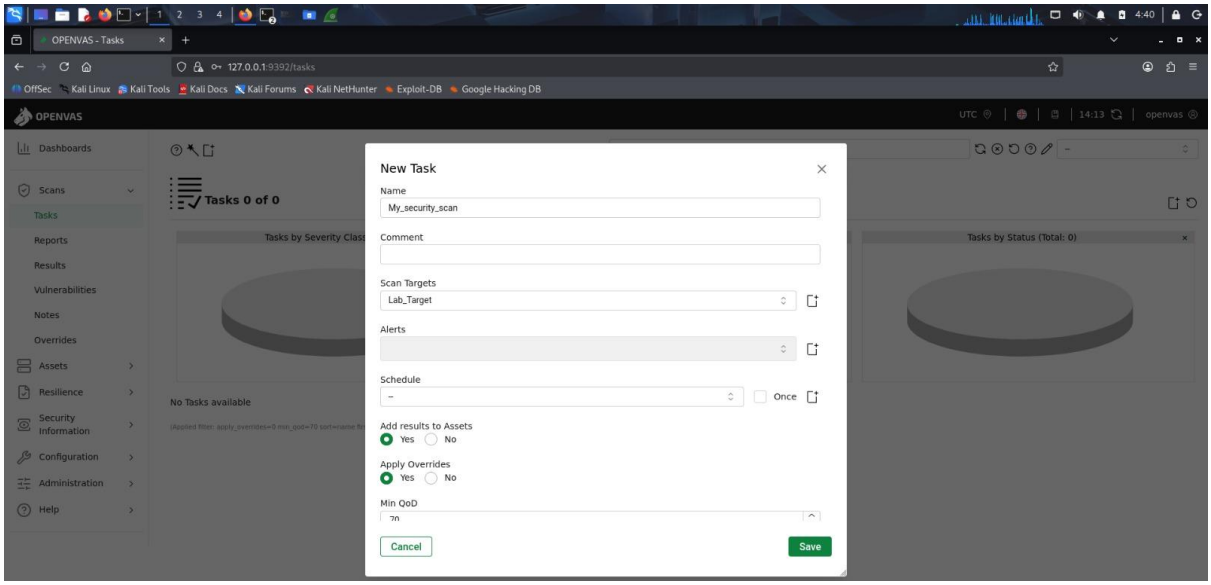


Step-9: Creating a New Scan Task

After configuring the target, a new scan task is created to perform the vulnerability assessment.

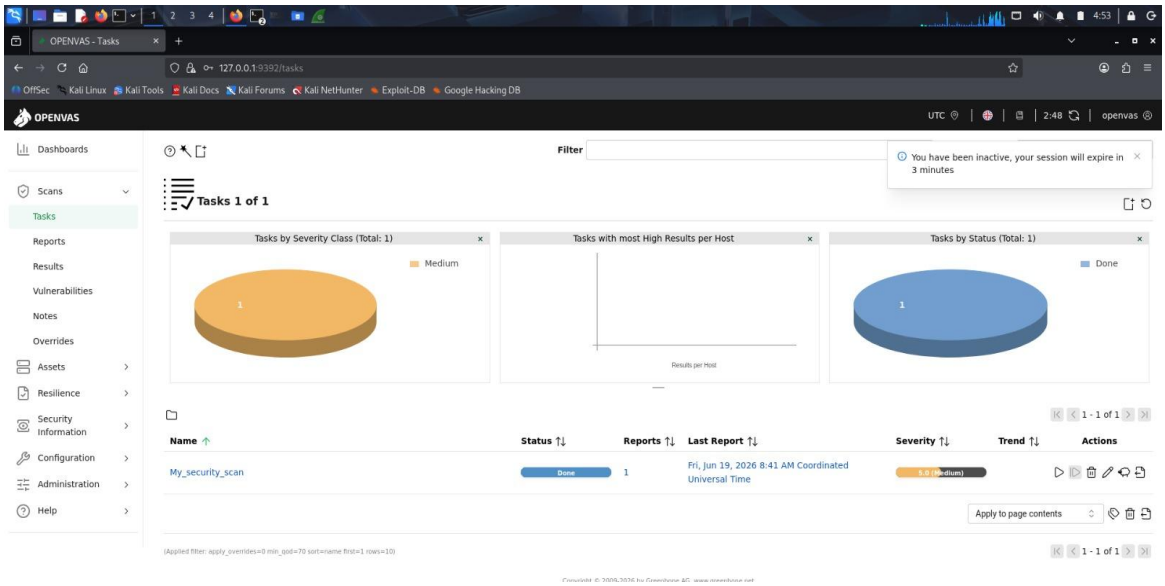
Configuration Steps:

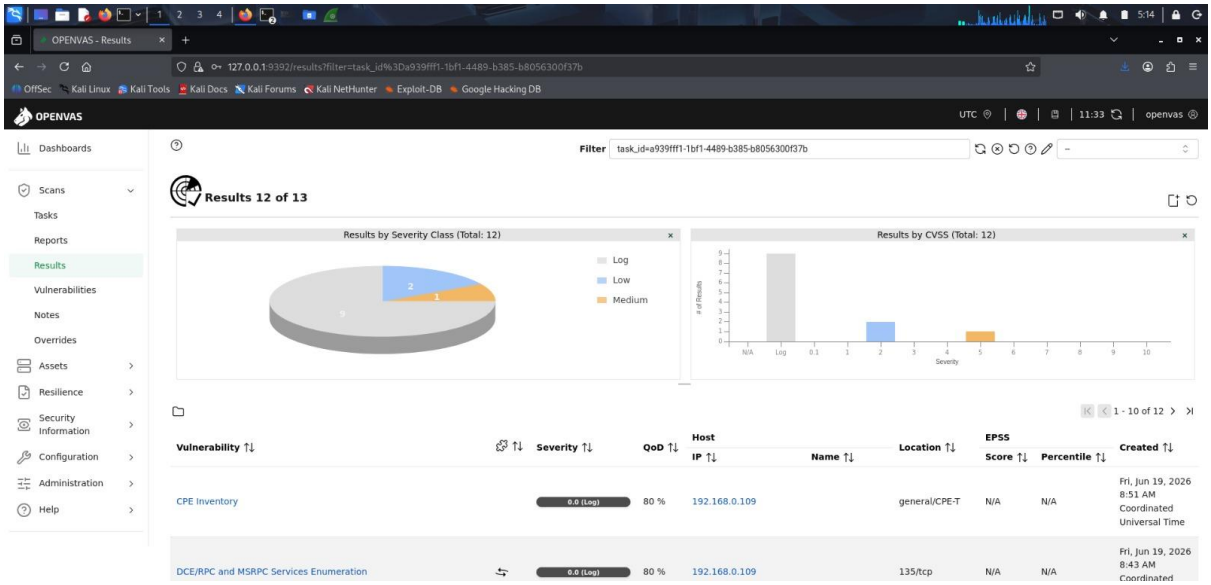
- Navigate to **Scans → Tasks**.
- Select **New Task**.
- Configure the following:
  - **Task Name:** My_security_scan
  - **Scan Target:** Lab_Target
- Save the task.
- Start the vulnerability scan by clicking the **Play (Start)** button.

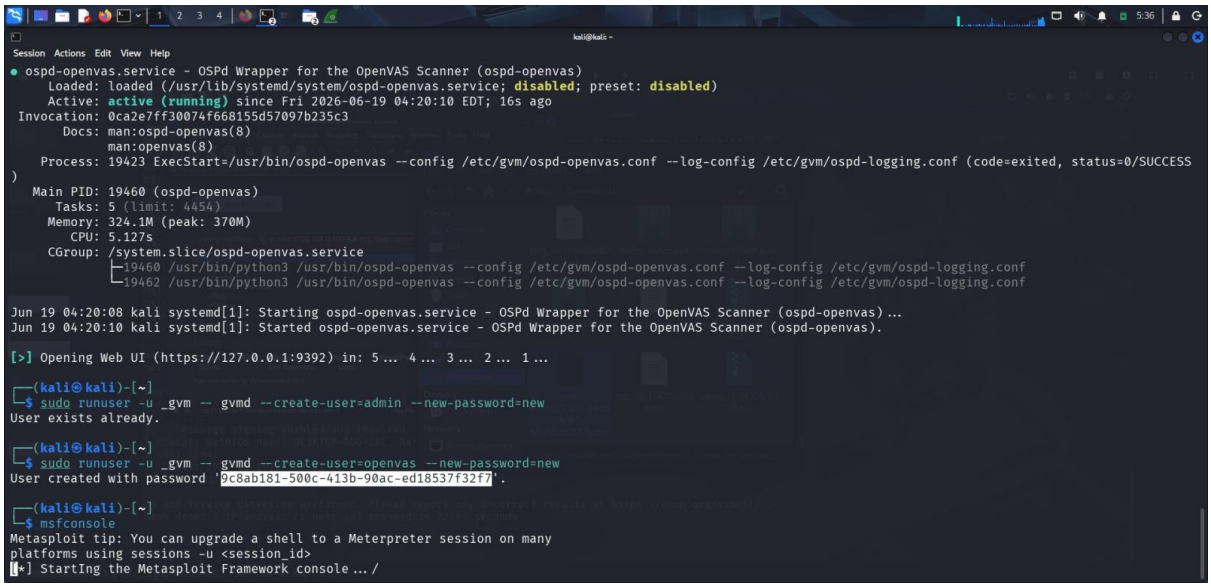


Step-10: Analyzing Scan Results and Generating Reports

After the vulnerability scan completes successfully, the task status changes to **Done**, indicating that the assessment has finished.



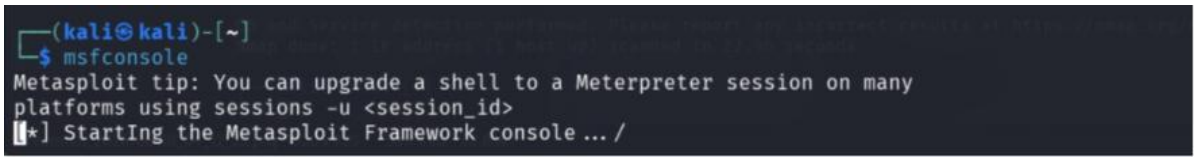




Vulnerability Verification (Metasploit Framework)

Step-1: Starting Metasploit Framework Console

The vulnerabilities and service information identified by OpenVAS were verified using Metasploit's auxiliary scanning modules.

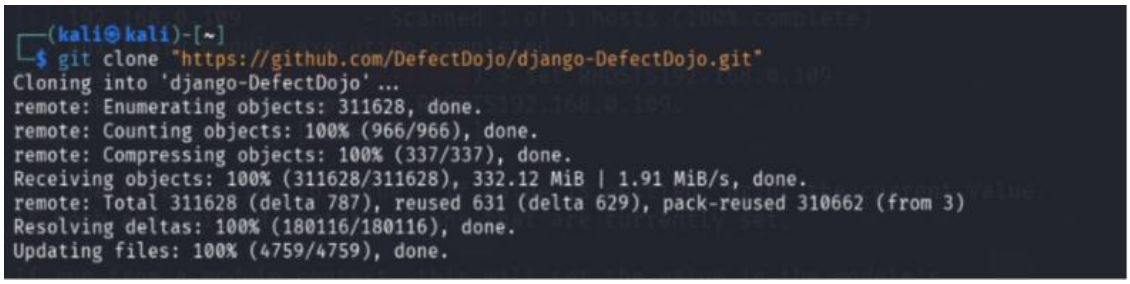


Step-2: Download DefectDojo and Start Docker (Terminal)

In the first image, the **DefectDojo** repository is cloned from GitHub, and the application is started using **Docker Compose**.

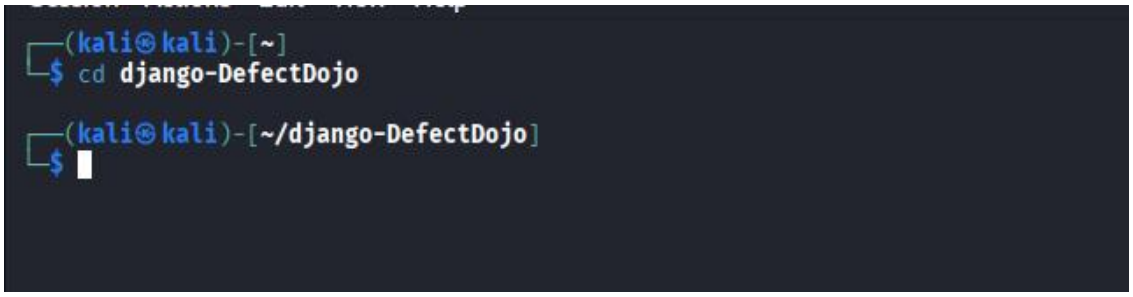
Clone the DefectDojo repository from GitHub

git clone https://github.com/DefectDojo/django-DefectDojo.git



Step-3: Navigate to the project directory

cd django-DefectDojo

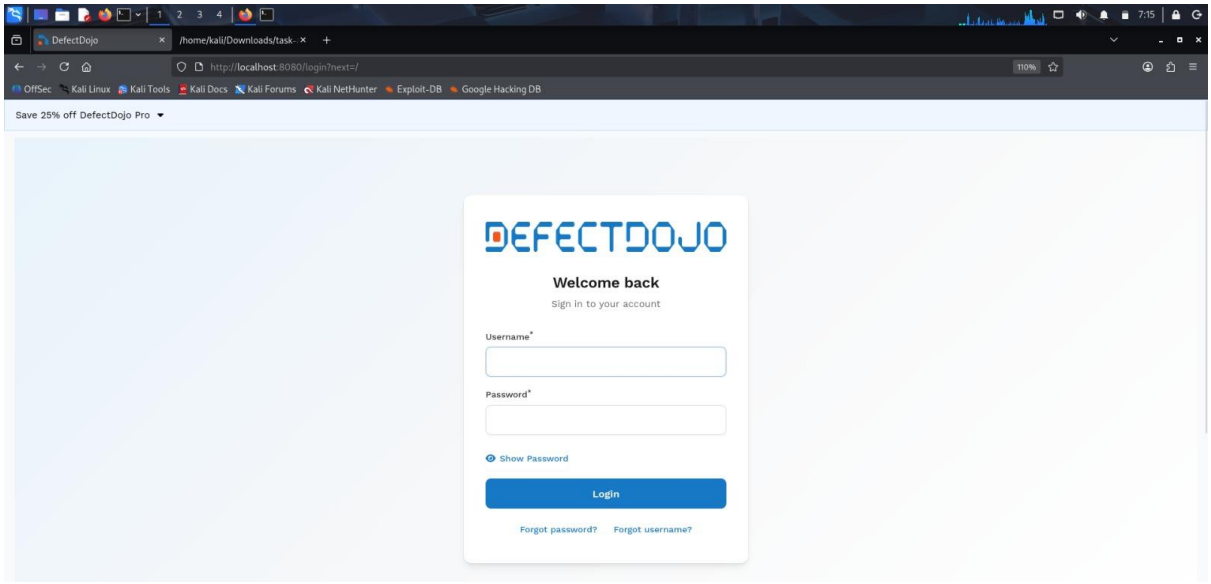


Step-4: Start the services in the background using Docker Compose

sudo docker-compose up -d

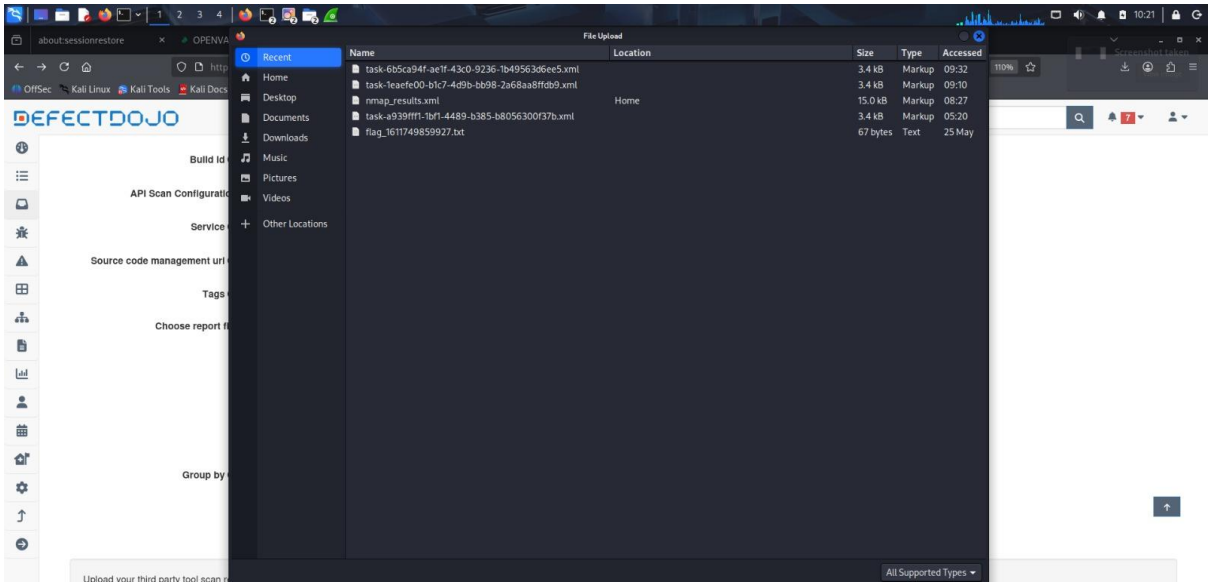
Step-5: Log in to the Dashboard (Browser Login)

In the second image, after the Docker containers have started successfully, open http://localhost:8080 in a web browser to access the DefectDojo login page. Then, log in using the assigned **Username** and **Password**.



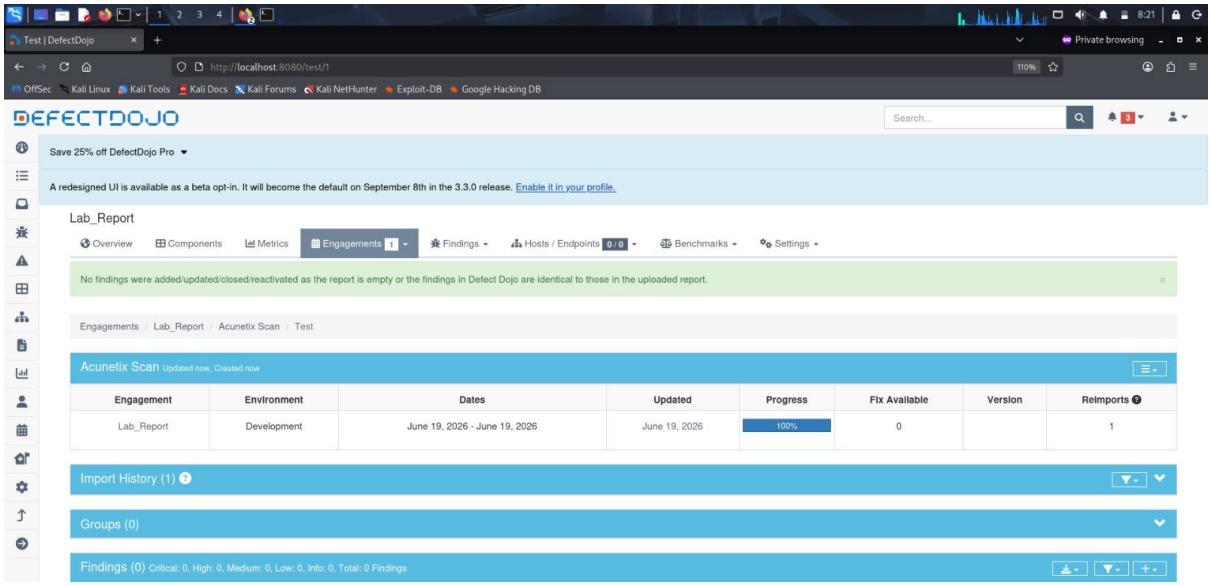
Step-6: Report File Upload

After logging in, this file selection window appears, allowing you to upload the vulnerability report file (such as .xml outputs from Nmap, OpenVAS, or Acunetix).



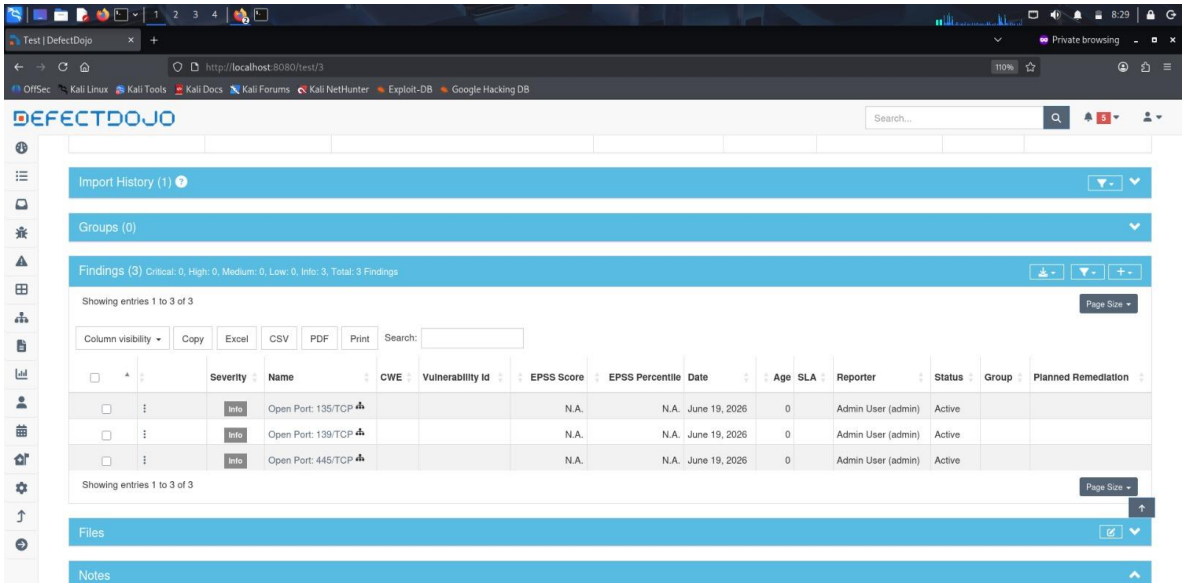
Step-7: Initial Engagement & Import Status Verification

This step shows the status of an active engagement (labeled as an *Acunetix Scan* here). A green banner notice confirms that the file was processed, though no new findings were added because the report was either empty or identical to a previous upload.



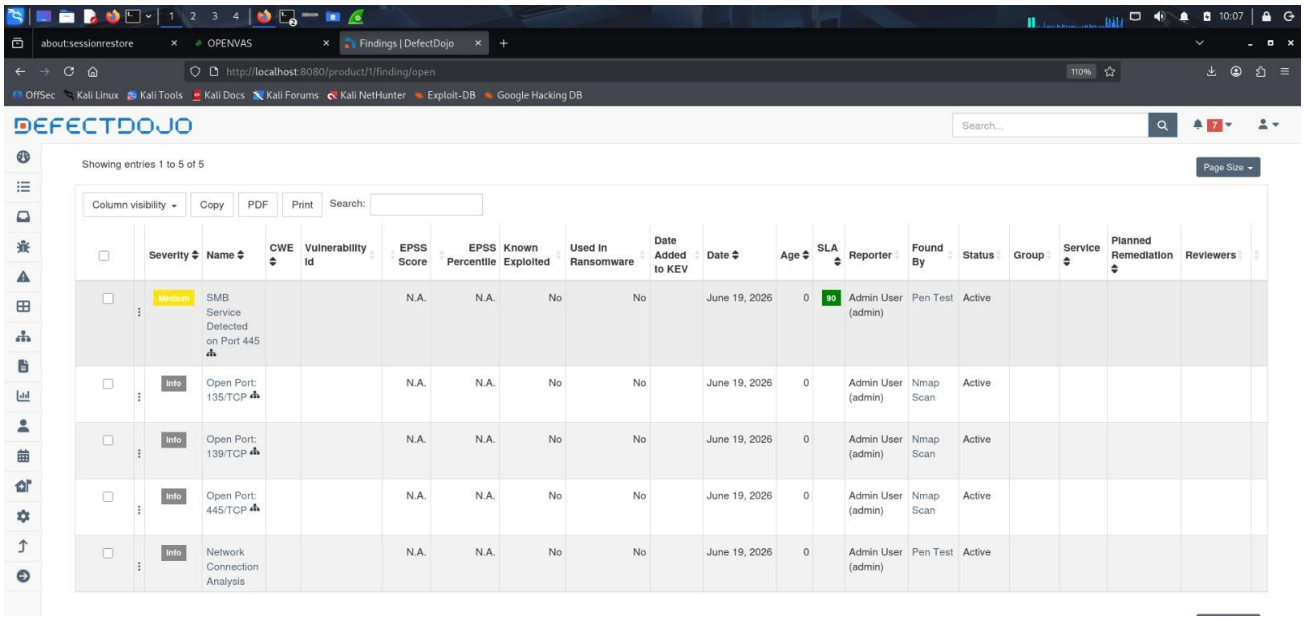
Step-8: Specific Scan Findings View

Once a report with actionable data is populated, this view displays the immediate findings from that specific import. In this view, 3 open ports (135/TCP, 139/TCP, 445/TCP) have been successfully logged as **Info** severity.



Step-9: Main Product Open Findings Dashboard

This is the comprehensive dashboard displaying all active open findings for the product. It aggregates vulnerabilities from multiple scans, currently showing 5 total findings (including 1 **Medium** severity SMB vulnerability alongside the **Info** logs).



Conclusion

The SecureGov Framework provides an integrated approach to security monitoring, vulnerability assessment, and risk management by combining multiple open-source cybersecurity tools into a unified workflow. It enhances the efficiency of identifying, analyzing, and managing security risks while supporting governance and compliance objectives. Overall, the framework demonstrates a practical, scalable, and cost-effective solution for improving an organization's cybersecurity posture and enabling proactive risk management.

References

1.Offensive Security. (2025). Kali Linux Documentation.

https://www.kali.org/docs/

2. Nmap Project. (2025). Nmap Reference Guide.

https://nmap.org/book/

3. Greenbone Networks. (2025). Greenbone Community Edition (OpenVAS/GVM) Documentation.

https://greenbone.github.io/docs/

4. Wireshark Foundation. (2025). Wireshark User's Guide.

https://www.wireshark.org/docs/

5. Rapid7. (2025). Metasploit Documentation.

https://docs.rapid7.com/metasploit/

6. DefectDojo Project. (2025). DefectDojo Documentation.
https://documentation.defectdojo.com/

7. OWASP Foundation. (2025). OWASP Testing Guide.

https://owasp.org/www-project-web-security-testing-guide/

8. National Institute of Standards and Technology (NIST). (2024). Cybersecurity Framework (CSF) 2.0.

https://www.nist.gov/cyberframework